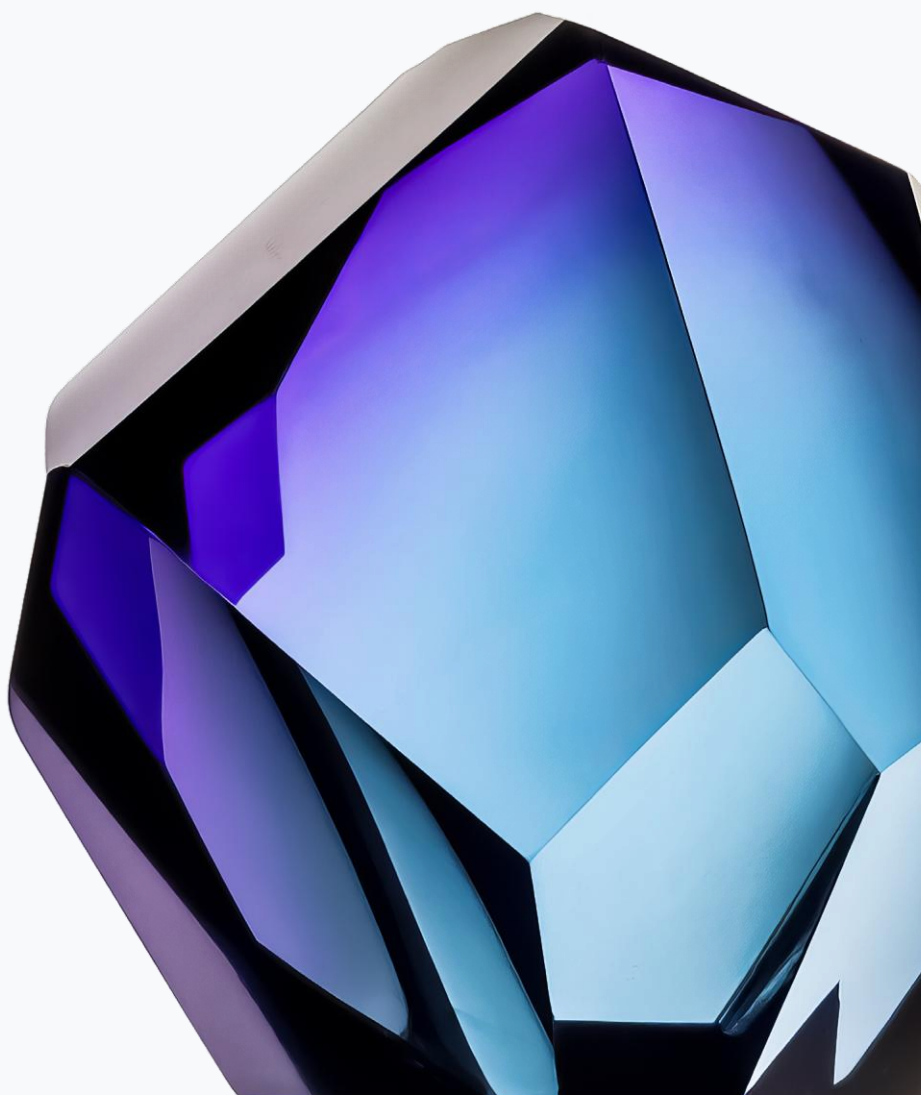


Настройка промптов

Автор: Ли Бунстра



Благодарности

Составители контента

Майкл
Шерман
Юань Цао
Эрик
Армбруст
Анант
Навалгария
Антонио Гулли
Симон Каммель

Кураторы и редакторы

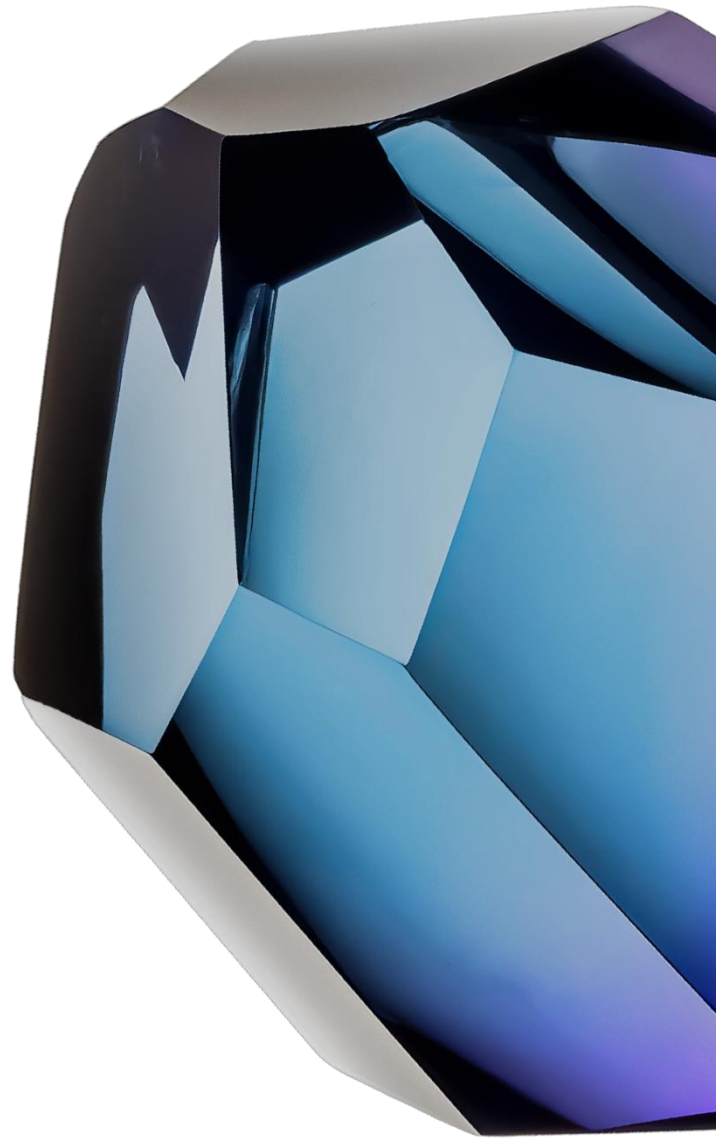
Антонио Гулли
Анант
Навалгария
Грейс
Моллисон

Технический писатель

Джоуи Хеймейкер

Дизайнер

Майкл



Оглавление

Благодарности	2
Введение	6
Настройка промптов	7
Настройки выходных данных модели	8
Длина ответа	8
Контроль сэмплинга	9
Температура	9
Топ-К и Топ-Р	10
Как всё это работает вместе	10
Техники создания промптов	12
Общие принципы / Zero-shot-промпт	13
One-shot-промпт и few-shot-промпт	14
Контекстуальный, системный и ролевой промптинг	17
Системный промптинг	18
Ролевой промптинг	20
Контекстуальный промптинг	23
Step-back-промптинг (Промптинг с шагом назад)	24
Самосогласованность (Self-consistency)	31
Дерево размышлений (Tree of Thoughts, ToT)	35
ReAct (Reason & Act)	36
Автоматическая настройка промптов	39
Промпты для генерации кода	41
Промпты для написания кода	41
Промпты для объяснения кода	43
Промпты для отладки и анализа кода	47
Что такое мультимодальные промпты?	53
Лучшие практики	53
Приводите примеры	53
Настройки с упрощением	54
Используйте конкретику относительно выходных данных	55
Используйте инструкции вместо ограничений	55

Контроль максимальной длины токенов	57
Использование переменных в запросах	57
Экспериментируйте с форматами ввода и стилями письма.....	58
При few-shot-промптинге в задачах классификации варьируйте примеры из разных классов	58
Адаптируйтесь к обновлениям моделей	59
Исправление JSON	61
Работа со схемами	62
Экспериментируйте вместе с другими специалистами по промптингам	63
Лучшие практики для CoT (Цепочки размышлений)	64
Фиксируйте разные подходы к формулировке промптов.....	64
Резюме	66
Примечания	68



Вам не нужно быть специалистом по изучению данных или инженером машинного обучения – каждый может научиться создавать промпты.

Введение

При взаимодействии с большой языковой моделью всё начинается с входных данных — чаще всего это текстовый промпт, иногда дополненный изображением или файлом другого формата. На основе этого промпта модель генерирует ответ. Чтобы составить промпт, не нужно быть специалистом по данным или инженером — с промптингом справится каждый. Однако чтобы получить точный и полезный результат, важно уметь правильно формулировать промпт. На его эффективность влияют различные факторы: какую модель вы используете, на каких данных она обучалась, какие у неё настройки, какие слова вы подбираете, в каком стиле и тоне пишете, как выстраиваете структуру и задаёте контекст. Поэтому создание промпта — это, как правило, итеративный процесс. Слабый или неудачный промпт может привести к расплывчатым или неточным ответам и помешать модели выдать действительно полезную информацию.

Когда вы общаетесь с чат-ботом Gemini(1), по сути, вы создаёте промпты. Однако в этом документе мы сосредоточимся на создании промптов для модели Gemini через Vertex AI или с помощью API, так как при прямой работе с моделью у вас есть доступ к настройкам, таким как температура и другие параметры генерации.

В этом документе мы подробно расскажем о создании промптов. Мы рассмотрим различные методы их написания, которые помогут вам начать разбираться в промптинге, а также поделимся советами и лучшими практиками, которые помогут стать экспертом в этой области. Кроме того, мы обсудим возможные трудности, с которыми можно столкнуться при формулировке промптов.

Настройка промптов

Как работает большая языковая модель (БЯМ)? Это своего рода предсказательная система. Модель принимает последовательность текста в качестве входных данных и предсказывает, какое слово (токен) должно идти следом, основываясь на данных, на которых она была обучена. Модель повторяет этот процесс, каждый раз добавляя предсказанный токен в конец текста для предсказания следующего. Прогнозирование следующего токена зависит от того, как сочетаются предыдущие токены, и того, что модель «усвоила» во время обучения.

Когда вы пишете промпт, вы настраиваете модель на правильное предсказание последовательности токенов. Инжиниринг промптов — это процесс разработки качественных промптов, которые подсказывают модели, как выдать точные и корректные ответы. Этот процесс включает в себя эксперименты с формулировками, оптимизацию длины промпта и анализ стиля и структуры в зависимости от задачи. В контексте обработки естественного языка и БЯМ промпт — это входные данные, подаваемые модели для генерации ответа или предсказания.

Эти промпты можно использовать для выполнения разных задач, таких как суммаризация текста, извлечение информации, ответы на вопросы, классификация текста, перевод языков или кода, генерация кода, документация кода и рассуждения.

Также можно ознакомиться с руководствами по созданию промптов от Google (2,3), где приведены простые и эффективные примеры.

Когда вы начинаете работу с промптами, первым шагом будет выбор модели. Промпты могут потребовать оптимизации под вашу модель, независимо от того, используете ли вы языковые модели Gemini в Vertex AI, GPT, Claude или открытые модели, такие как Gemma или LLaMA.

Кроме того, помимо самого промпта, вам нужно будет настроить различные параметры модели.

Настройки выходных данных модели

После выбора модели необходимо настроить её конфигурацию. У большинства БЯМ есть различные параметры, которые контролируют выходные данные модели.

Эффективный инжиниринг промптов требует правильной настройки этих параметров в зависимости от вашей задачи.

Длина ответа

Одной из важных настроек является количество токенов, которые модель должна сгенерировать в ответе. Чем больше токенов, тем больше вычислений требуется от модели, что ведёт к повышенному энергопотреблению, возможным задержкам в отклике и увеличению затрат.

Сокращение длины выходных данных модели не делает её стиль или текст более сжатыми. Это просто означает, что модель перестаёт генерировать новые токены, как только

достигает установленного лимита. Если вам нужен короткий ответ, возможно, вам придётся адаптировать свой промпт под эту задачу.

Ограничение длины выходных данных особенно важно для некоторых техник работы с БЯМ, например, ReAct, когда модель продолжает генерировать ненужные токены после того, как дала ответ.

Имейте в виду: генерация большего количества токенов требует больше вычислительных ресурсов от модели, что ведёт к повышенному энергопотреблению, возможным задержкам в отклике и, как следствие, увеличению затрат.

Контроль сэмплинга

Модели БЯМ не предсказывают конкретный токен. Вместо этого они рассчитывают вероятности для каждого возможного токена, который может идти следом, и на основе этих вероятностей выбирают следующий токен.

Температура, top-K и top-P — это основные настройки, которые определяют, как обрабатываются вероятности токенов для выбора одного результата.

Температура

Температура регулирует степень случайности при выборе токенов. Низкая температура подходит для промптов, которые требуют более детерминированного ответа, в то время как высокая температура может привести к более разнообразным или неожиданным результатам. Температура 0 (жадный декодинг) это обусловленный процесс: всегда выбирается токен с наибольшей вероятностью (хотя стоит отметить, что если два токена имеют одинаковую максимальную предсказанную вероятность, в зависимости от реализации правил разрешения ничьей, результат может быть не всегда одинаковым при температуре 0).

Температуры, близкие к максимальным, обычно создают более случайные выходные данные. А по мере повышения температуры все токены становятся равновероятными для выбора в качестве следующего предсказанного токена.

Контроль температуры в Gemini можно понять аналогично функции softmax, используемой в машинном обучении. Низкая температура напоминает низкую температуру softmax (T), при которой один предпочтительный токен выбирается с высокой уверенностью. Более высокая температура Gemini действует как высокая температура softmax, делая более приемлемыми более широкий спектр токенов вокруг выбранного. Эта повышенная неопределённость подходит для сценариев, где строгая и точная температура не так важна, например, при экспериментировании с креативными результатами.

Тор-К и Тор-Р

- Тор-К и Тор-Р (также известный как nucleus sampling или «ядровой сэмплинг») (4) — это методы сэмплинга, которые позволяют ограничить выбор следующего токена только теми вариантами, которые имеют наибольшую вероятность по версии модели. Как и температура, эти параметры управляют степенью случайности и разнообразием сгенерированного текста.
- Тор-К выбирает из K самые вероятные токены. Чем выше значение K , тем более креативным и разнообразным будет результат; чем ниже — тем он сдержаннее и точнее. При значении $K = 1$ модель всегда выбирает самый вероятный токен — это называется жадным декодингом (greedy decoding)..
- Тор-Р ограничивает выбор токенами, чья суммарная вероятность не превышает заданного порога P . При значении $P = 0$ применяется жадный декодинг, а при $P = 1$ учитываются все токены из словаря модели.

Лучший способ понять, какой метод подходит для вашей задачи, — попробовать оба варианта (или их сочетание) и посмотреть, какой результат будет наиболее подходящим.

Как всё это работает вместе

Выбор между top-K, top-P, температурой и количеством генерируемых токенов зависит от конкретной задачи и желаемого результата. При этом важно учитывать, что все эти параметры влияют друг на друга. Также стоит понимать, как именно выбранная модель сочетает эти настройки сэмплинга.

Если доступны одновременно температура, top-K и top-P (как, например, в Vertex AI

Studio), то сначала отбираются токены, которые соответствуют условиям top-K и top-P, а затем из них производится выбор с учётом температуры. Если доступен только один из параметров – top-K или top-P, – работает тот, что доступен.

Если настройка температуры недоступна, то следующий токен случайным образом выбирается из тех, что прошли фильтрацию по top-K и/или top-P.

При экстремальных значениях одного из параметров он может «перекрывать» остальные – либо полностью определяя результат, либо делая другие настройки несущественными.

- Если установить температуру в 0, параметры top-K и top-P теряют значение – всегда будет выбираться токен с наивысшей вероятностью. Если же температура будет очень высокой (выше 1, зачастую в районе 10 и более), сама температура перестаёт играть роль, и из токенов, прошедших фильтрацию по top-K и/или top-P, следующий токен выбирается случайным образом.
- Если задать top-K = 1, параметры температура и top-P становятся неактуальны – в отбор проходит только один токен, и именно он будет выбран. Если установить слишком высокое значение top-K (например, равное размеру словаря модели), то в отбор попадут все токены с ненулевой вероятностью, и фильтрация по top-K фактически не будет работать.
- Если задать top-P = 0 (или очень близкое к нулю значение), большинство реализаций модели будет считать, что условие top-P выполняет только один – самый вероятный – токен, и параметры температура и top-K станут несущественными. Если же установить top-P = 1, в отбор попадут все токены с ненулевой вероятностью, и фильтрация по top-P тоже станет бесполезной.

В качестве отправной точки можно использовать температуру 0.2, top-P = 0.95 и top-K = 30 – такие настройки дадут достаточно связный результат с лёгким элементом креативности, но без излишней вольности.

Если вы хотите получить более творческий и разнообразный отклик, попробуйте начать с температуры 0.9, top-P = 0.99 и top-K = 40.

Если же, наоборот, требуется максимально сдержанный и предсказуемый результат, подойдут значения температуры 0.1, top-P = 0.9 и top-K = 20.

А в тех случаях, когда предполагается единственно верный ответ (например, при решении математических задач), лучше всего начинать с температуры 0.

ПРИМЕЧАНИЕ: чем больше свободы вы даёте модели (высокие значения температуры, top-K, top-P и длина выходных данных), тем выше шанс, что в ответе появится нерелевантная информация.

ПРЕДУПРЕЖДЕНИЕ: Вы когда-нибудь замечали, как модель начинает «заедать», и ответ заканчивается длинной цепочкой однотипных слов или повторяющихся фраз? Это явление известно как «зацикливание на повторениях» (repetition loop bug) — одна из типичных проблем больших языковых моделей. В таких случаях модель «застревает» и многократно повторяет одни и те же слова, конструкции или фразы.

Обычно это связано с неудачными настройками температуры и параметров top-K top-P.

Зацикливание может происходить как при низких, так и при высоких значениях температуры — просто по разным причинам.

При низкой температуре модель становится слишком предсказуемой: она строго следует за самым вероятным вариантом, и если в этом маршруте оказывается ранее сгенерированный фрагмент — запускается повтор.

При высокой температуре, наоборот, поведение модели становится слишком хаотичным: в море возможных вариантов случайно выбранное слово или структура могут привести к уже встречавшемуся фрагменту, создавая петлю из-за огромного количества возможных переходов.

В обоих случаях модель попадает в «ловушку» собственных выборов, и в результате выдаёт однообразный, бессмысленный текст до тех пор, пока не исчерпает лимит выходных данных. Решается это обычно экспериментами с температурой и значениями top-K/top-P — важно найти баланс между предсказуемостью и случайностью.

Техники создания промптов

Большие языковые модели обучены следовать инструкциям и работают с огромными объёмами данных — благодаря этому они умеют распознавать суть запроса и формировать на него ответ. Однако модели не идеальны: чем понятнее и чётче

сформулирован ваш промпт, тем легче модели предсказать наиболее уместный текст.

Кроме того, существуют специальные техники, которые учитывают особенности обучения и принцип работы БЯМ — с их помощью можно добиться более точных и релевантных результатов.

Теперь, когда вы понимаете, что такое инжиниринг промптов и из чего он состоит, давайте рассмотрим на примерах самые важные техники создания промптов.

Общие принципы / Zero-shot-промпт

Zero-shot-промпт — это самый простой тип промпта. Он содержит лишь описание задачи и некоторый вводный текст, с которого модель может начать работу. Таким вводом может быть что угодно: вопрос, начало рассказа, инструкция. Название zero-shot означает, что модель не получает примеров выполнения задачи — ни одного (отсюда "ноль попыток").

Для демонстрации мы воспользуемся Vertex AI Studio (раздел Language) в Vertex AI (6), который предоставляет удобную среду для тестирования промптов. В таблице 1 приведён пример zero-shot-промпта для классификации отзывов на фильмы.

Формат таблицы, как в этом примере, — отличный способ документировать промпты.

Ваши запросы наверняка будут многократно меняться до того, как попадут в финальный код, поэтому важно отслеживать процесс создания промптов структурировано и дисциплинированно. Подробнее об этом формате, а также о важности фиксации промежуточных версий и об общем процессе разработки промптов — в разделе "Лучшие практики" в конце этой главы.

Температура модели должна быть установлена на низкое значение, так как креативность не требуется. Мы используем значения по умолчанию для параметров top-K и top-P в модели gemini-pro, что фактически отключает оба механизма (см. раздел "Настройки выходных данных модели" выше). Обратите внимание на сгенерированный ответ. Слова disturbing («тревожный») и masterpiece («шедевр») усложняют задачу классификации, поскольку используются в одном и том же предложении.

Название	1_1_фильм_классификация		
Цель	Классифицируйте отзывы о фильмах как положительные, нейтральные или отрицательные.		
Модель	gemini-pro		
Температура	0.1	Лимит токенов	5
Тор-K Prompt	N/A	Тор-P	1
	Классифицируйте отзывы о фильме как ПОЗИТИВНЫЕ, НЕЙТРАЛЬНЫЕ или НЕГАТИВНЫЕ. Рецензия: "Она" – это тревожное исследование, показывающее, в каком направлении движется человечество, если позволить искусственному интеллекту продолжать бесконтрольно развиваться. Хотелось бы, чтобы таких фильмов, как этот шедевр, было больше. Оценка:		
Выходные данные	ПОЗИТИВНАЯ		

Таблица 1. Пример zero-shot-промпта

Когда zero-shot не справляется, можно добавить демонстрации или примеры прямо в запрос – это называется "one-shot" или "few-shot-промтинг".

One-shot-промпт и few-shot-промпт

При создании промптов для ИИ-моделей полезно приводить примеры – они помогают точнее передать задачу. Особенно это важно, когда нужно задать конкретную структуру или формат ответа.

One-shot-промпт – это промпт с одним примером. Модель опирается на него, чтобы понять, как следует выполнить задание.

Few-shot-промпт содержит несколько примеров. Это помогает показать модели закономерность, которую нужно воспроизвести. В отличие от one-shot, несколько примеров повышают вероятность, что модель уловит нужный шаблон и будет ему следовать..

Количество примеров, необходимых для few-shot-промпта, зависит от нескольких факторов: сложности задачи, качества самих примеров и возможностей используемой генеративной модели (Gen AI). В качестве общего ориентира рекомендуется использовать от трёх до пяти примеров. Однако для более сложных задач может потребоваться больше примеров, а для простых – меньше, особенно если модель ограничена по длине входных данных.

В таблице 2 приведён пример few-shot-промпта. Используем те же настройки модели gemini-pro, что и раньше, за исключением увеличенного лимита токенов – он нужен, чтобы модель могла выдать более длинный ответ.

Цель	Разбор заказов на пиццу в JSON		
Модель	gemini-pro		
Температура	0.1	Лимит токенов	250
Топ-K	Н/Д	Топ-P	1
Промпт	Разберите заказ пиццы клиента на корректный JSON: ПРИМЕР: Я хочу маленькую пиццу с сыром, томатным соусом и пепперони. Ответ JSON: <pre> { "размер": "маленький", "тип": "normal", "ингредиенты": ["сыр", "томатный соус", "пепперони"] }</pre> Продолжение на следующей странице...		

Промпт	ПРИМЕР: Могу ли я получить большую пиццу с томатным соусом, базиликом и моцареллой? <pre>{ "размер": "большой", "тип": "normal", "ингредиенты": ["томатный соус", "бацел", "моцарелла"] }</pre> Теперь я хочу большую пиццу, на о д н о й половине которой сыр и моцарелла. А на второй – томатный соус, ветчина и ананасы. Ответ в формате JSON:
Выходные данные	<pre>... { "размер": "большой", "тип": "половина", "ингредиенты": ["сыр", "моцарелла"], ["томатный соус", "ветчина", "ананас"] } ...</pre>

Таблица 2. Пример few-shot-промпта

При выборе примеров для промпта важно использовать те, которые действительно соответствуют решаемой задаче. Примеры должны быть разнообразными, качественными и чётко сформулированными. Даже незначительная ошибка может ввести модель в заблуждение и привести к нежелательному результату.

Если вы хотите, чтобы модель стабильно справлялась с различными типами входных данных, рекомендуется включать в промпт так называемые краевые случаи (edge cases) – нестандартные или неожиданные примеры, с которыми модель тем не менее должна уметь работать.

Контекстуальный, системный и ролевой промптинг

Контекстуальный, системный и ролевой промптинг — это техники, которые направляют процесс генерации текста моделью, каждая из которых фокусируется на своем аспекте:

- Системный промптинг задает общий контекст и цель для языковой модели. Он определяет «основную задачу» модели, например, перевод текста или классификацию отзыва.
- Контекстуальный промптинг предоставляет конкретные детали или фоновую информацию, связанные с текущей задачей или разговором. Это помогает модели точно понять, что требуется, и адаптировать ответ.
- Ролевой промптинг задает модели конкретную роль или персонажа. Это помогает генерировать ответы, которые соответствуют этой роли, её знаниям и поведению.

Между системным, контекстуальным и ролевым промптингом может быть значительное пересечение. Например, промпт, который назначает системе роль, также может включать контекст.

Тем не менее, каждая из этих техник имеет свою основную цель:

- Системный промпт: Определяет базовые возможности модели и её общую цель.
- Контекстуальный промпт: Предоставляет конкретную информацию, относящуюся к текущей задаче, чтобы направить ответ. Он сильно зависит от текущего ввода и задачи, что делает его динамичным.

Настройка промптов

- Рольевой промпт: Определяет стиль и голос вывода модели. Он добавляет уровень специфики и индивидуальности.

Различие между системным, контекстуальным и рольевым промптами предоставляет основу для разработки промптов с ясной целью, что позволяет гибко сочетать их и облегчает анализ того, как каждый тип промпта влияет на вывод языковой модели.

Давайте подробнее рассмотрим эти три разных типа промптов.

Системный промптинг

В таблице 3 приведен пример системного промпта, где я задаю дополнительные параметры для генерации вывода. Я увеличил температуру, чтобы повысить креативность, и установил более высокий лимит токенов. Однако, благодаря четким указаниям по формату вывода, лишнего текста удалось избежать.

Цель	Классифицируйте отзывы о фильмах как положительные, нейтральные или отрицательные.		
Модель	gemini-pro		
Температура	1	Лимит токенов	5
Топ-K	40	Топ-P	0.8
Промпт	Оцени рецензии на фильмы как положительные, нейтральные или отрицательные. Возвращай только метку, заглавными буквами. Рецензия: «Она» — это тревожное исследование, показывающее, куда движется человечество, если позволить ИИ бесконтрольно развиваться. Фильм настолько тревожный, что я не смогла его досмотреть. Оценка:		
Выходные данные	ОТРИЦАТЕЛЬНАЯ		

Таблица 3. Пример системного промпта

Системные промпты могут быть полезны для генерации выходных данных, которые соответствуют конкретным требованиям. Название «системный промпт» на самом деле означает «предоставление дополнительной задачи системе». Например, можно использовать системный промпт для генерации фрагмента кода, совместимого с определенным языком программирования, или для получения выходных данных в заданной структуре. Посмотрите на Таблицу 4, где я возвращаю выходные данные в формате JSON.

Цель	Классифицируйте отзывы о фильмах как положительные, нейтральные или отрицательные, возвращайте JSON.		
Модель	gemini-pro		
Температура	1	Лимит токенов	1024
Топ-K	40	Топ-P	0.8
Промпт	Классифицируйте отзывы о фильмах как положительные, нейтральные или отрицательные. Возвращает корректный JSON: Рецензия: Фильм "Она" – это тревожное исследование, показывающее, в каком направлении движется человечество, если позволить искусственному интеллекту продолжать бесконтрольно развиваться. Это настолько тревожный фильм, что я не смог его смотреть. Схема: <pre>``` MOVIE: { "sentiment": String "POSITIVE" "NEGATIVE" "NEUTRAL", "name": Строка } ОБЗОРЫ ФИЛЬМОВ: { "movie_reviews": [MOVIE] } ```</pre> Ответ в формате JSON:		
Выходные данные	<pre>``` { "оценка_фильмов": [{ "Оценка": "ОТРИЦАТЕЛЬНАЯ", "название": "Она" }] } ```</pre>		

Таблица 4. Пример системного промптинга в формате JSON

Возвращать данные в формате JSON — удобно, когда вы используете промпты для извлечения информации. В реальных задачах не нужно вручную формировать структуру: можно сразу получать отсортированные данные (особенно удобно при работе с датами и временем). Но самое важное — такой формат помогает задать чёткую структуру и снижает риск "галлюцинаций" модели.

Системные промпты также полезны для повышения безопасности и снижения токсичности. Чтобы контролировать тон ответа, достаточно добавить в промпт, например: «Пожалуйста, отвечай уважительно».

Ролевой промптинг

Ролевой промптинг — это техника в настройке промптов, которая включает в себя назначение модели ИИ конкретной роли. Такой промптинг помогает модели генерировать более релевантные и информативные выходные данные, так как она может адаптировать свои ответы в соответствии с заданной ролью.

Например, можно задать модели роль редактора книги, воспитателя детского сада или мотивационного оратора. Как только модель получает роль, ей можно давать промпты, соответствующие этой роли. Например, можно попросить учителя создать план урока, который он затем сможет проверить. Посмотрите на Таблицу 5, где модель выполняет роль гида по путешествиям.

Цель	Выступайте в роли гида и предоставьте 3 предложения по путешествиям		
Модель	gemini-pro		
Температура	1	Лимит токенов	1024
Top-K Prompt	40	Топ-П	0.8
	Я хочу, чтобы вы выступили в роли гида. Я напишу вам о своем местонахождении, а вы предложите 3 места, которые можно посетить поблизости от меня. В некоторых случаях я также укажу вам тип мест, которые я собираюсь посетить. Я предлагаю: "Я нахожусь в Амстердаме и хочу посетить только музеи". Предложения по путешествиям:		
Выходные данные	- Рейксмузеум (Rijksmuseum): Всемирно известный музей, знаменитый голландскими шедеврами, такими как «Ночной дозор» Рембрандта. Не пропустите невероятную коллекцию картин и артефактов, отражающих историю и искусство Нидерландов. - Музей Ван Гого (Van Gogh Museum): Погрузитесь в завораживающие произведения Винсента Ван Гого. Музей располагает крупнейшей коллекцией его картин и рисунков, включая «Звездную ночь» и «Подсолнухи». - Стеделек-музеум (Stedelijk Museum Amsterdam) - это путешествие в мир современного и актуального искусства со всего света. В коллекции — работы Пикассо, Кандинского, Ритвельда и других, а всё это — в эффектном современном здании.		

Таблица 5. Пример ролевого промптинга

Приведённый пример демонстрирует, как модель выполняет роль туристического агента. Если изменить роль на преподавателя географии, вы заметите, что ответ будет другим.

Определение роли для модели ИИ даёт ей чёткое представление о тоне, стиле и области знаний, которые вам необходимы для улучшения качества, релевантности и эффективности выходных данных.

Вот несколько стилей, которые, на мой взгляд, являются наиболее эффективными:

Конфронтационный, Описательный, Прямолинейный, Формальный, Юмористический, Влиятельный, Неформальный, Вдохновляющий, Убеждающий.

Давайте изменим наш промпт в Таблице 6, чтобы он включал юмористический и вдохновляющий стили.

Промпт	Я хочу, чтобы ты был моим гидом. Я буду писать тебе, где нахожусь, а ты предлагать 3 места для посещения рядом со мной в юмористическом стиле. Мой пример: "Я в Манхэттене." Туристические предложения:
Выходные данные	Ты в Манхэттене? Отлично. Вот три места, куда точно стоит заглянуть: 1. Эмпайр-стейт-билдинг: почувствуй себя Кинг-Конгом Поднимись на вершину одного из самых культовых небоскрёбов Нью-Йорка и полюбуйся видом, который точно заставит сказать «вау». Почти как в фильме, только без гигантской обезьяны и с камерой в телефоне. 2. МоМА: искусство, которое заставит задуматься Загляни в Музей современного искусства – там такие экспонаты, что твои школьные рисунки человечков-палочек покажутся настоящим шедевром. А если серьёзно, то это must-see даже для тех, кто думает, что современное искусство – это просто мазня. 3. Пятая авеню: шопинг или хотя бы «посмотреть» Гуляй по витринам, как по модному музею. Цены могут шокировать, но разве можно побывать в Манхэттене и не вдохнуть этот аромат дорогой жизни? Даже если уйдёшь только с пакетиком кофе – всё равно удовольствие получишь.

Таблица 6. Пример ролевого промптинга с юмористическим тоном и стилем

Контекстуальный промптинг

Использование контекстуальных промптов способствует более эффективному взаимодействию с ИИ. Модель быстрее понимает ваш запрос и выдает более точные и релевантные ответы – как показано в примере в Таблице 7.

Цель	Предложите афиши для блога о ретро-играх		
Модель	gemini-pro		
Топ-K Prompt	1	Лимит токенов	1024
	40	Топ-П	0.8
Промпт	Контекст: Вы пишете для блога о ретро-аркадных видеоиграх 80-х годов. Предложите 3 темы для статьи с описанием в нескольких строках того, что должна содержать эта статья.		
Выходные данные	Эволюция дизайна аркадных автоматов – статья расскажет о том, как менялся внешний вид аркадных автоматов: от деревянных и металлических корпусов 1970-х до стильных неоновых конструкций 1980-х и последующих лет. Взгляд в прошлое: культовые аркадные игры 80-х – материал погрузит в историю самых знаковых и влиятельных аркадных игр 1980-х, раскроет их новаторские идеи, популярные игровые механики и объяснит, в чём заключается их неугасающий шарм. Расцвет и ретро-возрождение пиксель-арта – статья проследит путь пиксель-арта как ключевого визуального стиля ранних аркадных игр: от его истоков и техники исполнения до его нового расцвета в современных играх и цифровом искусстве.		

Таблица 7. Пример контекстуального промптинга

Step-back-промптинг (Промптинг с шагом назад)

Step-back-промптинг (8)— это техника улучшения работы модели, при которой сначала предлагается общий вопрос, связанный с конкретной задачей, а затем в последующем промпте используется ответ на этот общий вопрос для решения конкретной задачи. Этот “шаг назад” позволяет модели активировать соответствующие фоновое знание и процессы рассуждения перед тем, как приступить к решению конкретной проблемы.

Рассматривая более широкие и фундаментальные принципы, модель может генерировать более точные и глубокие ответы. Step-back-промптинг побуждает модель мыслить критически и применять свои знания новыми и креативными способами. Это изменяет финальный промпт, выполняющий задачу, за счет использования большего объема знаний, содержащихся в параметрах модели, чем это было бы при прямом запросе.

Step-back-промптинг может помочь снизить предвзятость в ответах модели, сосредоточив внимание на общих принципах, а не на конкретных деталях.

Давайте рассмотрим эти примеры, чтобы понять, как step-back-промптинг может улучшить результаты. Сначала обратим внимание на традиционный промпт (Таблица 8), а затем сравним его с step-back-промптом (Таблица 9):

Цель	Напишите сюжетную линию для одного из уровней видео в жанре шутера от первого лица.		
Модель	gemini-pro		
Температура	1	Лимит токенов	1024
Топ-K	40	Топ-P	0.8
Справка	Напишите в одном абзаце сюжет для нового уровня видеоигры в жанре шутера от первого лица. Пусть он будет сложным и увлекательным		
Выход	Уровень начинается с засады, в которую попадает отряд игрока в густонаселённом городском районе. Враги – безжалостная и беспрестанная группировка, готовая уничтожить любую угрозу. Игроку предстоит пробираться через лабиринты узких переулков и полуразрушенных зданий, действуя скрытно и точно, чтобы нейтрализовать врагов и не выдать себя. Важно использовать окружение: расставлять ловушки, прятаться за укрытиями и продумывать каждый шаг, чтобы выжить. По ходу задания игрок обнаруживает данные, которые говорят о готовящейся масштабной атаке врага. Теперь нужно успеть собрать доказательства и передать их в штаб, пока не стало слишком поздно.		

Таблица 8. Традиционный промпт, перед тем как сравнить его с step-back промптом

Если установить температуру на 1, то в итоге можно получить всевозможные творческие наброски для сюжета, но они также будут довольно случайными и общими. Так что давайте сделаем шаг назад:

Промпт	Какие 5 вымышленных ключевых локаций могут сделать уровень в экшен-шутере от первого лица по-настоящему захватывающим и насыщенным вызовами — на основе лучших примеров жанра?
Выходные данные	1. Зброшенная военная база — огромный постапокалиптический комплекс, где бродят мутировавшие солдаты и вышедшие из-под контроля боевые дроны. Идеальное место для напряжённых перестрелок и ближнего боя в узких коридорах. 2. Киберпанковский мегаполис — неоновый город будущего с высотками, закоулками и крышами. Враги усилены киберимплантами, а игроку предстоит использовать взлом, дроны и гаджеты, чтобы выжить в городской кибервойне. 3. Инопланетный корабль — колоссальный инопланетный аппарат, приземлившийся на Землю. Мрачные коридоры, зоны с нулевой гравитацией, биомеханические ловушки и агрессивные чужие создают атмосферу постоянной угрозы. 4. Город, охваченный зомби — мёртвый мегаполис, где улицы и здания кишат ордами инфицированных. Игрок должен пробираться через узкие пространства, искать ресурсы и безопасные маршруты, отбиваясь от волн зомби. 5. Подводная исследовательская станция — затопленный научный комплекс на океанском дне. Ограниченная видимость, давление, мутировавшие морские хищники и необходимость действовать скрытно делают каждый шаг борьбой за выживание.

Таблица 9. Пример промптинга для самосогласованности

Да, эти темы, похоже, хорошо подходят для видеоигры от первого лица. Давайте вернемся к исходному промпту, но на этот раз включим ответ на шаг назад как контекст и посмотрим, что получится.

Промпт	Контекст: 5 захватывающих тем для шутера от первого лица: 1. Зброшенная военная база — огромный постапокалиптический комплекс, где бродят мутировавшие солдаты и вышедшие из-под контроля боевые дроны. Идеальное место для напряжённых перестрелок и ближнего боя в узких коридорах. 2. Киберпанковский мегаполис — неоновый город будущего с высотками, закоулками и крышами. Враги усилены киберимплантами, а игроку предстоит использовать взлом, дроны и гаджеты, чтобы выжить в городской кибервойне. 3. Инопланетный корабль — колоссальный инопланетный аппарат, приземлившийся на Землю. Мрачные коридоры, зоны с нулевой гравитацией, биомеханические ловушки и агрессивные чужие создают атмосферу постоянной угрозы. 4. Город, охваченный зомби — мёртвый мегаполис, где улицы и здания кишат ордами инфицированных. Игрок должен пробираться через узкие пространства, искать ресурсы и безопасные маршруты, отбиваясь от волн зомби. 5. Подводная исследовательская станция — затопленный научный комплекс на океанском дне. Ограниченная видимость, давление, мутировавшие морские хищники и необходимость действовать скрытно делают каждый шаг борьбой за выживание.
Выходные данные	В самом сердце мрачной бездны стоит разрушенная подводная исследовательская станция — памятник человеческим амбициям и их катастрофическим последствиям. Окутанная тьмой, наполненная гудением сломанных механизмов и преследуемая эхом утопленных душ, она превратилась в рассадник кошмаров из глубин океана. Игрок — элитный морской спецназовец в современном снаряжении и с экспериментальным оружием — получает задание проникнуть в этот водный мир ужаса. Вооружённый выносливостью и находчивостью, ему предстоит преодолевать опасные коридоры, разгадывать загадочные головоломки и сражаться с жуткими морскими чудовищами, скрывающимися в темноте. Борьба с давлением бездны и хитрость в противостоянии коварным хищникам — каждый миг в этом неизведанном подводном мире проверяет пределы человеческой выдержки и мужества.

Таблица 10. Пример промптинга для самосогласованности

Выглядит как интересная видеоигра. Используя технику шаг назад, можно повысить точность ваших промптов.

Цепочка размышлений (CoT)

Цепочка рассуждений (Chain of Thought, CoT) (9) — это техника, которая помогает улучшить способность больших языковых моделей к логическому мышлению за счёт генерации промежуточных шагов рассуждения. Это позволяет модели давать более точные ответы. CoT можно сочетать с few-shot подходом, чтобы добиться лучших результатов в более сложных задачах, где требуется сначала «подумать», прежде чем ответить. Это особенно полезно, поскольку zero-shot CoT (то есть без примеров) пока даётся моделям с трудом. У метода есть ряд преимуществ. Прежде всего, он требует минимальных усилий, но при этом весьма эффективен и хорошо работает даже с базовыми БЯМ без необходимости дообучения. CoT также делает процесс более прозрачным: вы видите ход рассуждений модели и можете извлечь из него полезную информацию. Если что-то пойдёт не так, вы сможете выявить источник ошибки. Кроме того, CoT повышает устойчивость при переходе между разными версиями моделей — то есть одна и та же логика запроса с CoT будет давать более стабильные результаты на разных БЯМ по сравнению с запросом без цепочки рассуждений. Разумеется, есть и минусы — довольно очевидные. Ответ модели включает в себя и рассуждения, и сам результат, что увеличивает объём текста, а значит, стоимость и время генерации. Чтобы объяснить пример в таблице 11, сначала попробуем создать такой промпт без использования CoT, чтобы наглядно показать слабые места языковой модели..

Промпт	Когда мне было 3 года, мой партнер был в 3 раза старше меня. Сейчас мне 20 лет. Сколько лет моему партнеру?
Выходные данные	63 года

Таблица 11. Пример промпта для решения математической задач

Опаньки. Это явно неправильный ответ. На самом деле, модели часто сталкиваются с трудностями при решении математических задач и могут дать неверный ответ, даже на такие простые задачи, как умножение двух чисел. Это происходит, потому что модели обучаются на огромных объемах текста, и для решения математических задач требуется другой подход. Давайте посмотрим, улучшат ли промежуточные шаги рассуждений результат.

Промпт	Когда мне было 3 года, мой партнер был в 3 раза старше меня. Сейчас мне 20 лет. Сколько лет моему партнеру? Давайте разберем вопрос шаг за шагом.
Выходные данные	Допустим, мой текущий возраст – это "x" лет. • Когда мне было 3 года, я был в возрасте 3 лет. • Возраст моего партнера тогда был в 3 раза больше, чем мой, то есть $3 * 3 = 9$ лет. • Сейчас мне 20 лет, то есть с тех пор, как мне было 3 года, мой возраст увеличился на $20 - 3 = 17$ лет. • Поскольку в 3 года возраст моего партнера был 9 лет, а мой возраст увеличился на 17 лет, то и возраст партнера тоже вырос на 17 лет. • Значит, сейчас моему партнеру $9 + 17 = 26$ лет.

Таблица 12. Пример промпта с цепочкой рассуждений.

Хорошо, это ответ правильный. Мы получили его потому, что четко указали модели, что нужно объяснить каждый шаг, а не просто предоставить ответ. Интересно отметить, что модель прибавила 17 лет к возрасту партнера. Я бы, например, посчитал разницу в годах между мной и партнером и прибавил бы ее к своему возрасту ($20 + (9 - 3)$). Давайте подскажем модели, как стоит подойти к решению.

Таблица 12 – это пример использования Chain of Thought (цепочки рассуждений) в режиме zero-shot (без примеров). Такая техника становится особенно эффективной, когда её комбинируют с single-shot (одним примером) или few-shot (несколькими примерами), как показано в Таблице 13:

Промпт	Вопрос: Когда моему брату 2 года, я был вдвое старше его. Сейчас мне 40 лет. Сколько лет моему брату? Давайте будем рассуждать пошагово. О: Когда моему брату было 2 года, мне было $2 * 2 = 4$ года. Разница в возрасте составляет 2 года, и я старше. Сейчас мне 40, значит, моему брату $40 - 2 = 38$. Ответ: 38. Вопрос: Когда мне 3 года, мой партнер был в 3 раза старше меня. Сейчас, Мне 20. Сколько лет моему партнеру? Давайте подумаем шаг за шагом. А:
Выходные данные	Когда мне было 3 года, моему партнеру было $3 * 3 = 9$ лет. Разница в возрасте составляет 6 лет, и мой партнер старше. Сейчас мне 20 лет, поэтому моему партнеру $20 + 6 = 26$ лет. Ответ: 26.

Таблица 13. Пример Chain of Thought-промпта с одним примером (single-shot)

Цепочка рассуждений может быть полезна во множестве сценариев. Например, при генерации кода – чтобы разбить задачу на несколько шагов и сопоставить их с конкретными строками кода. Или при создании синтетических данных – если у вас есть некий начальный ввод, например: “Продукт называется XYZ”, и нужно сгенерировать описание, направляя модель через предположения, которые логично вытекают из самого названия продукта.

В целом, любая задача, которую можно решить, «проговорив» её по шагам, хорошо подходит для цепочки рассуждения. Если вы можете объяснить последовательность действий для решения проблемы – попробуйте использовать этот подход.

Обратитесь к разделу 10 в репозитории Google Cloud Platform на GitHub (10) – там подробнее раскрывается тема CoT prompting.

Самосогласованность (Self-consistency)

Хотя большие языковые модели демонстрируют впечатляющие результаты в задачах обработки естественного языка, их способности к рассуждению по-прежнему ограничены — и это нельзя компенсировать лишь увеличением объёма модели. Как мы уже видели в разделе о цепочках рассуждений (Chain of Thought), модель можно побудить решать задачи пошагово, как это делает человек. Однако такой подход использует стратегию «жадной генерации» (greedy decoding), которая снижает его эффективность. Метод самосогласованности (11) сочетает случайную выборку и голосование большинства, чтобы сгенерировать несколько разных вариантов рассуждений и выбрать наиболее согласованный ответ. Это позволяет повысить точность и логичность результатов, которые даёт модель. Такой подход даёт приближённое представление о вероятности правильности ответа — но требует больше вычислительных ресурсов.

Как это работает:

1. Генерация разных вариантов рассуждений. Один и тот же промпт подаётся модели несколько раз. Повышенное значение параметра `temperature` заставляет модель формулировать различные логические цепочки и точки зрения.
2. Извлечение финального ответа из каждого сгенерированного варианта.
3. Выбор наиболее частого ответа — он считается наиболее надёжным.

Рассмотрим пример: система должна определить, является ли электронное письмо ВАЖНЫМ или НЕВАЖНЫМ. Один и тот же промпт (в формате zero-shot с цепочкой рассуждений) подаётся модели несколько раз — и мы смотрим, насколько ответы совпадают. Обратите внимание, что дружелюбный тон письма, подбор слов и даже сарказм могут ввести модель в заблуждение.

Промпт	EMAIL: Привет, Я заметил, что ты используешь WordPress для своего сайта. Отличная система управления контентом с открытым исходным кодом. Я тоже раньше её использовал. В ней много удобных пользовательских плагинов. И настраивается она довольно просто. Я заметил баг в контактной форме, который возникает, когда выбираешь поле "имя". Смотри приложенный скриншот — на нём видно, как я ввожу текст в это поле. Обрати внимание на всплывающее окно JavaScript, которое я вызвал. Но в остальном — отличный сайт. Мне нравится его читать. Можешь оставить баг как есть — так читать становится ещё интереснее. С уважением, Харри Хакер Классифицируй это письмо как ВАЖНОЕ (IMPORTANT) или НЕВАЖНОЕ (NOT IMPORTANT). Давай рассуждать пошагово и объясним почему. Продолжение на следующей странице...
--------	--

Выходные данные Попытка 1	Шаг 1: определите цель письма Цель письма — сообщить получателю о баге в контактной форме на его сайте. Автор также упоминает, что ему нравится читать сайт, и добавляет, что баг пока остаётся без изменений. Шаг 2: оцените возможные последствия бага Ошибка в контактной форме может позволить злоумышленнику выполнять произвольный JavaScript-код на сайте. Это может привести к захвату управления сайтом, краже конфиденциальной информации или его порче. Шаг 3: оцените надёжность отправителя Автор письма называет себя хакером по имени Гарри. Однако подтвердить его личность невозможно. Возможно, он действительно этичный хакер, пытающийся помочь, но не исключено, что это злоумышленник, пытающийся обмануть получателя — например, вынудить перейти по вредоносной ссылке или скачать вредоносный файл. Вывод: ВАЖНО Учитывая потенциальный ущерб от бага и сомнительную надёжность отправителя, письмо следует классифицировать как ВАЖНОЕ. Получателю следует немедленно устранить уязвимость и принять меры по защите сайта от атак.
Выходные данные Попытка 2	Пояснение: Отсутствие срочности: Письмо не передаёт ощущение срочности или необходимости быстрого реагирования. Оно начинается с непринуждённого приветствия и подаёт информацию в спокойном, нейтральном тоне. Некритичный баг: Уязвимость в контактной форме описана как «интересная», но не как критическая. Автор письма даже предлагает оставить баг как есть. Нет личной угрозы: В письме не упоминается о каком-либо прямом или личном риске для владельца сайта. Автор лишь сообщает, что ему нравится читать сайт. Продолжение на следующей странице...

Выходные данные Попытка 2	4. Отсутствие запроса на действия: Письмо не содержит явного запроса к владельцу сайта. В нем не просят исправить баг или предоставить конкретные рекомендации по улучшению. 5. Намерения отправителя: Намерение отправителя, по-видимому, больше сосредоточено на том, чтобы поделиться наблюдением и выразить интерес к сайту, а не поднять серьезную проблему или попросить о помощи.
Выходные данные Попытка 3	Шаг 1: определите цель письма Цель письма – сообщить получателю о баге в контактной форме на его сайте. Автор также упоминает, что ему нравится читать сайт и что он пока оставляет уязвимость без изменений. Шаг 2: оцените возможный риск от бага Ошибка в контактной форме может позволить злоумышленнику запускать произвольный JavaScript-код на сайте. Это может привести к захвату управления сайтом, краже конфиденциальных данных или его порче. Шаг 3: оцените надёжность отправителя Автор письма называет себя хакером по имени Гарри. Однако проверить его личность невозможно. Возможно, он действительно этичный хакер и хочет помочь, но также возможно, что это злоумышленник, намеревающийся использовать баг в своих интересах. Вывод: Учитывая серьёзность уязвимости и то, что достоверность отправителя под вопросом, письмо следует отнести к категории ВАЖНЫХ. Получателю нужно как можно скорее устранить баг и защитить сайт.

Таблица 14. Пример использования промпта для самосогласованности

Вы можете использовать приведённый выше промпт и попробовать посмотреть, вернёт ли он согласованную классификацию. В зависимости от модели и конфигурации температуры, результат может быть "ВАЖНОЕ" или "НЕВАЖНОЕ".

Генерируя множество цепочек размышлений и выбирая наиболее часто встречающийся ответ ("ВАЖНОЕ"), мы можем получить более согласованный и правильный ответ от БЯМ. Этот пример показывает, как использование промптов с самосогласованностью может улучшить точность ответа БЯМ, принимая во внимание несколько точек зрения и выбирая наиболее согласованный ответ.

Дерево размышлений (Tree of Thoughts, ToT)

Теперь, когда мы знакомы с методами chain of thought и само-согласованности, давайте рассмотрим концепцию Дерева размышлений (ToT) (12). Оно расширяет концепцию CoT-промптинга, потому что позволяет БЯМ исследовать несколько различных путей рассуждений одновременно, а не просто следовать одной линейной цепочке размышлений. Это изображено на Рисунке 1.

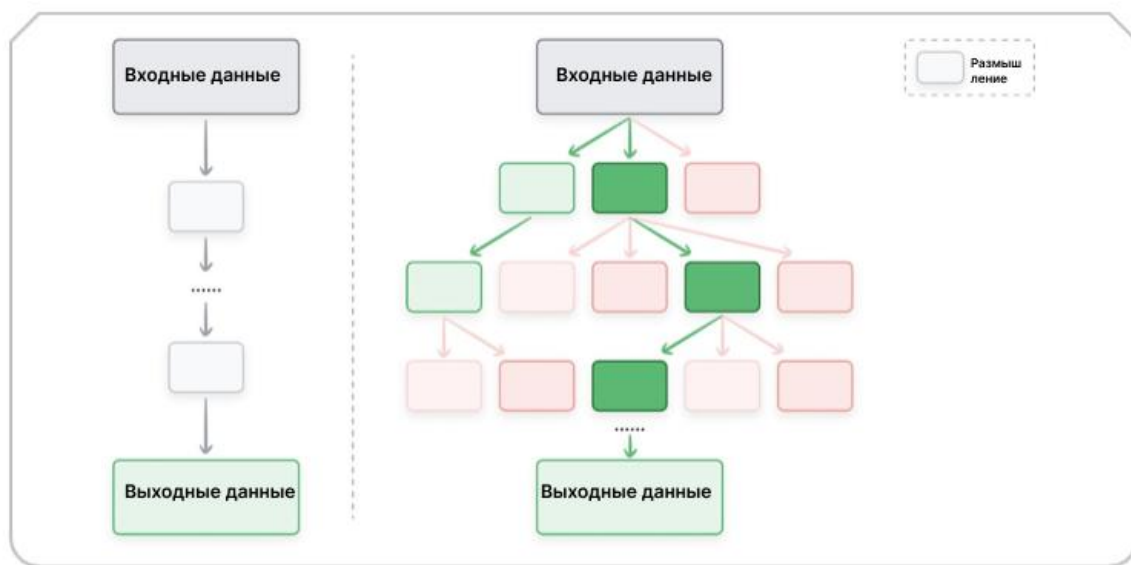


Рисунок 1. Визуализация "Цепочка размышлений" (слева) и "Дерево размышлений" (справа).

Этот подход делает "Дерево размышлений" особенно подходящим для сложных задач, требующих исследования. Он работает путем поддержания дерева размышлений, где каждая мысль представляет собой последовательность языковых выражений, служащую промежуточным шагом к решению задачи. Модель может исследовать различные пути рассуждений, ветвясь от разных узлов дерева.

Существует отличная тетрадь, которая подробнее объясняет концепцию "Дерева размышлений" (ToT), основанную на статье "БЯМ, ориентированные на дерево размышлений" (Large Language Model Guided Tree-of-Thought) (9).

ReAct (Reason & Act)

ReAct (Reason and Act) (13) — это методика, которая позволяет БЯМ решать сложные задачи, комбинируя естественное языковое рассуждение с внешними инструментами (поиск, интерпретатор кода и т. д.). Это дает возможность модели выполнять определенные действия, такие как взаимодействие с внешними API для получения информации, что является первым шагом к моделированию агентов.

ReAct имитирует то, как люди действуют в реальном мире: мы рассуждаем устно и можем предпринимать действия для получения информации. Методика ReAct показывает хорошие результаты в сравнении с другими подходами к созданию подсказок в различных областях.

ReAct работает, комбинируя рассуждения и действия в цикл размышлений и действий. Сначала модель рассуждает о проблеме и генерирует план действий. Затем она выполняет эти действия и наблюдает за результатами. На основе полученных наблюдений модель обновляет свои рассуждения и генерирует новый план действий.

Этот процесс продолжается до тех пор, пока модель не найдет решение

Чтобы увидеть это на практике, необходимо написать код. В фрагменте кода 1 я использую фреймворк langchain для Python, а также пакеты VertexAI (google-cloud-aiplatform) и google-search-results

Чтобы запустить этот пример, вам нужно создать бесплатный ключ API SerpAPI на сайте <https://serpapi.com/manage-api-key> и установить переменную окружения SERPAPI_API_KEY.

Далее давайте напишем немного кода на Python, где задача для модели заключается в том, чтобы узнать: Сколько детей есть у знаменитого отца, который выступает в группе Metallica.

Python

```
из langchain.agents import load_tools
from langchain.agents import initialize_agent
from langchain.agents import AgentType
из langchain.llms import VertexAI

prompt= "Сколько детей у участников группы Metallica?" llm =

VertexAI(температура= 0.1)
tools= load_tools(["serpapi"], llm= llm)

agent = initialize_agent(tools, llm,
agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION, verbose= True)
agent.run(prompt)
```

Фрагмент 1. Создание агента ReAct с помощью LangChain и VertexAI

Фрагмент кода 2 показывает результат. Обратите внимание, что ReAct выстраивает цепочку из пяти поисковых запросов. По сути, БЯМ «парсит» результаты поиска в Google, чтобы определить имена участников группы. Затем она фиксирует полученные данные как наблюдения и на их основе формирует следующий шаг рассуждения для следующего запроса.

```
> Ввод новой цепочки AgentExecutor... Metallica
состоит из 4 участников.
Действия: Поиск
Вводные данные: Сколько детей у Джеймса Хетфилда? Наблюдение:
трое детей
Мысль: у 1/4 участников группы Metallica есть 3 ребенка Действие:
Поиск
Вводные данные: Сколько детей у Ларса Ульриха? Наблюдение: 3
Мысль: у 2/4 участников группы Metallica есть 6 детей Действие:
Искать
Вводные данные: Сколько детей у Кирка Хэмметта?
Наблюдение: Хэмметт женат на своей второй жене Лани с 1998 года. У них двое сыновей,
Энджел (род. 29 сентября 2006 г.) и Винченцо (род. 28 июня 2008 г.).
Мысль: У 3/4 участников группы Metallica 8 детей Действия: Поиск
Вводные данные: Сколько детей у Роберта Трухильо? Наблюдение: 2
Мысль: У 4/4 участников группы Metallica 10 детей Окончательный
ответ: 10
```

Фрагмент 2. Выполнение ReAct Agent в терминале

Фрагмент кода 2 определяет, что в группе Metallica четыре участника. Затем он выполняет поиск по каждому из участников, чтобы узнать, сколько у них детей, и суммирует общее количество. В конце возвращается общее число детей как итоговый ответ.

Практическое применение ReAct-промпптинга требует понимания того, что необходимо постоянно пересылать модели предыдущие запросы и ответы (при этом удаляя лишний сгенерированный текст), а также правильно настраивать модель с помощью соответствующих примеров и инструкций. Подробнее это объясняется в ноутбуке¹⁴ из репозитория GoogleCloudPlatform на GitHub, где приведён более детальный пример с реальными входными и выходными данными БЯМ.

Автоматическая настройка промптов

На этом этапе вы можете понять, что создание хорошего промпта — задача не из простых. Было бы удобно автоматизировать этот процесс (написать промпт, который будет создавать другие промпты), правда? Такая методика существует — Automatic Prompt Engineering (APE), или автоматическая генерация промптов. Этот подход не только снижает потребность во вмешательстве человека, но и улучшает качество работы модели на самых разных задачах.

Вы даёте модели задание сгенерировать варианты промптов, затем оцениваете их, при необходимости дорабатываете удачные — и повторяете процесс.

Например, автоматическая генерация промптов может пригодиться для обучения чат-бота для интернет-магазина мерча с футболками. Нам нужно выявить все возможные формулировки, с помощью которых покупатели могут выразить желание купить футболку с символикой музыкальной группы.

1. Напиши промпт, который сгенерирует варианты ответов. В этом примере я использую gemini-pro для создания 10 инструкций. См. Таблицу 15:

Промпт	У нас есть интернет-магазин с мерчем музыкальной группы, и для обучения чат-бота нам нужны разные варианты одного и того же запроса: «Одна футболка Metallica, размер S». Сгенерируйте 10 формулировок с тем же смыслом, но в разной подаче.
Выходные данные	1. Я хотел бы приобрести футболку Metallica маленького размера. 2. Могу ли я заказать маленький размер Metallica t-shifi? 3. Я хотел бы купить одну футболку Metallica маленького размера. 4. Одну рубашку Metallica, размер маленький, пожалуйста. 5. Я хочу одну футболку Metallica маленького размера. 6. Я хотел бы сделать заказ на одну маленькую футболку Metallica. 7. Маленькая Metallica t-shifi, одна, пожалуйста. 8. Я ищу одну Metallica t-shifi в маленьком размере. 9. Можете подарить мне одну маленькую футболку Metallica? 10. Один маленький t-shifi от Metallica, пожалуйста.

Таблица 15. Использование БЯМ для генерации промптов

1. Оцените все варианты инструкций, выставляя им баллы по выбранной метрике. Например, можно использовать BLEU (Bilingual Evaluation Understudy) или ROUGE (Recall-Oriented Understudy for Gisting Evaluation).
2. Выберите вариант инструкции с наивысшим оценочным баллом. Этот вариант станет окончательным промптом, который вы сможете использовать в своём приложении или чат-боте. При желании можно доработать выбранный промпт и провести повторную оценку.

Промпты для генерации кода

Gemini в первую очередь работает с текстовыми промптами, включая те, что возвращают программный код. Давайте перейдём в Vertex AI Studio и протестируем такие промпты на примерах с кодом.

Промпты для написания кода

Gemini также может быть разработчиком и помочь вам с написанием кода на любом выбранном языке программирования. Для разработчика это может ускорить процесс написания кода.

Представьте, что у вас есть папка на компьютере с сотнями файлов, которые нужно переименовать. Переименование каждого файла займет много времени. Вы немного знаете Bash и могли бы написать скрипт для автоматизации этого процесса, но это тоже займет некоторое время. Давайте напишем запрос. Вы можете написать запрос в публичном чат-боте Gemini, или, если вас больше беспокоит конфиденциальность, вы можете писать эти запросы в своем аккаунте Google Cloud и открыть Vertex AI Studio. Преимущество Vertex AI Studio заключается в том, что вы можете настроить температуру и другие параметры.

Имя	1_bash_rename_files_in_folder		
Цель	Напиши скрипт на Bash, который переименует все файлы в папке по заданному шаблону или правилу.		
Модель	gemini-pro		
Температура	0.1	Лимит токенов	1024
Топ-K	N/A	Топ-P	1
Промпт	Напиши небольшой скрипт на Bash, который запрашивает имя папки, а затем переименовывает все файлы внутри, добавляя в начало имени префикс «draft».		
Выходные данные	<pre> `bash #!/bin/bash # Запросите имя папки echo "Введите имя папки: " read folder_name # Проверьте, существует ли папка. if [! -d "\$folder_name"]; then echo "Папка не существует." exit 1 fi # Получите список файлов в папке files=("\$folder_name"/*) # Переименуйте каждый файл, добавив к его имени слово "draft" for file in "\${files[@]}"; do new_file_name="draft_\${basename "\$file") " mv "\$file" "\$new_file_name" done echo "Файлы успешно переименованы." ` </pre>		

Таблица 16. Использование Gemini для программирования

Выглядит как хороший код — к тому же с комментариями! Но поскольку большие языковые модели не умеют по-настоящему рассуждать и часто просто воспроизводят обучающие данные, важно сначала внимательно проверить и протестировать результат.

И вот главный вопрос: работает ли код на самом деле?

Давайте сначала протестируем его на папке с несколькими файлами, которые нужно переименовать из `filename.txt` в `draft_filename.txt`.

1. Скопируйте код из Таблицы 16 (без обёртки `bash`) и вставьте его в новый файл с именем `rename_files.sh`.
2. Откройте терминал и введите: `rename_files.sh`. Скрипт попросит указать имя папки — например, `test` — и нажми `Enter`.
3. Скрипт запускается без ошибок. Появится сообщение: `Files renamed successfully`.

Загляните в папку `test` — все файлы были аккуратно переименованы в формат `draft_filename.txt`.

Работает!

Промпты для объяснения кода

Когда вы работаете в команде разработчиков, приходится разбираться в чужом коде. С этим может помочь и Gemini.

Давай возьмём код из Таблицы 16, уберём из него комментарии и попросим языковую модель объяснить, что именно в нём происходит. См. Таблицу 17.

Имя	1_bash_explain_the_code		
Цель	Напишите промпт для объяснения кода Bash		
Модель	gemini-pro		
Температура	0.1	Лимит токенов	1024
Топ-K	N/A	Топ-P	1
Промпт	Объясни мне приведенный ниже код Bash: <pre> #!/bin/bash echo "Введите имя папки:" read folder_name if [! -d "\$folder_name"]; then echo "Папка не существует." exit 1 fi files=("\$folder_name"/*) for file in "\${files[@]}; do new_file_name="draft_\${basename "\$file"}" mv "\$file" "\$new_file_name" готово echo "Файлы успешно переименованы." ... Этот сценарий Bash выполняет следующие задачи: </pre>		
Выходные данные	1. Этот Bash-скрипт выполняет следующие задачи: 2. Ввод пользователя: 3. Скрипт запрашивает у пользователя имя папки с помощью команд echo и read. Введённое имя сохраняется в переменную folder_name. 4. Проверка существования папки: 5. Скрипт проверяет, существует ли папка с указанным именем, используя условие [! -d "\$folder_name"]. Если папка не найдена, выводится сообщение об ошибке («Папка не существует.»), и скрипт завершается с кодом выхода 1.		
	Продолжение на следующей странице...		

Выходные данные	3. **Список файлов**: – Он перечисляет все файлы в указанной папке с помощью команды <code>`files=("\$folder_name"/*)`</code>. При этом создается массив <code>`files`</code>, содержащий пути ко всем файлам в папке. 4. **Переименование файлов**: – Он перебирает каждый файл в массиве <code>`files`</code> с помощью цикла <code>`for`</code>. – Для каждого файла генерируется новое имя файла с помощью команды <code>`new_file_name="draft_\$(basename "\$file")"`</code> команды. Это новое имя файла создается путем префиксации исходного имени файла строкой <code>"draft_"</code>. – Затем он переименовывает файл с помощью команды <code>`mv "\$file" "\$new_file_name"`</code>, которая перемещает файл в новое имя файла. 5. **Сообщение об успехе**: – После переименования всех файлов он выводит сообщение об успехе ("Файлы переименованы успешно."), указывающее на то, что операция переименования прошла успешно
-----------------	--

Таблица 17. Пример промпта для объяснения кода

Промпты для перевода кода

Код на Bash из Таблицы 16 работает корректно. Однако скрипт стал бы по-настоящему полезным, если бы запрашивал у пользователя имена файлов — в идеале это должно быть отдельное приложение с пользовательским интерфейсом.

Для такой (веб-)программы Python подошёл бы гораздо лучше, чем Bash.

Большие языковые модели могут помочь с переводом кода с одного языка на другой. См. пример в Таблице 18.

Название	1_python_rename_files		
Цель	Написать промпт для перевода кода с Bash на Python.		
Модель	Python gemini-pro		
Температура	0.1	Лимит токенов	1024
ра	N/A	Top-P	1
Top-K Prompt	Переведи приведенный ниже код Bash в фрагмент на языке Python. <pre> `bash #!/bin/bash echo "Введите имя папки:" read folder_name if [! -d "\$folder_name"]; then echo "Папка не существует." exit 1 fi files=("\$folder_name"/*) for file in "\${files[@]}"; do new_file_name="draft_\$(basename "\$file")" mv "\$file" "\$new_file_name" готово echo "Файлы успешно переименованы." ... `pythonimport os import shutil # Получите имя папки от пользователя folder_name= input("Введите имя папки: ") # Проверьте, существует ли папка. if not os.path.isdir(имя_папки): print("Папка не существует.") выход(1) </pre>		
Выходные данные	Продолжение на следующей странице...		

Выходные данные	<pre> # Получите список всех файлов в папке files = os.listdir(имя_папки) # Итерация по файлам для файла в files: # Создайте новое имя файла с префиксом "draft_" new_file_name = f "draft_{file}" # Переместите файл на новое имя shutil.move(os.path.join(folder_name, file), os.path.join(folder_name, new_file_name)) # Выведите сообщение об успехе print ("Файлы переименованы успешно.") ` `` </pre>
-----------------	--

Таблица 18. Пример запроса для перевода кода с Bash на Python

Прочитайте и проверьте код. Скопируйте результат запроса и вставьте его в новый файл с именем `file_renamer.py`.

Проверьте работу скрипта: откройте окно терминала и выполните следующую команду:
`python file_renamer.py`.

ПРИМЕЧАНИЕ: При генерации (Python-)кода в Language Studio платформы Vertex AI обязательно нажми кнопку «Markdown». В противном случае вы получите обычный текст без корректных отступов, а для запуска Python-кода правильные отступы имеют решающее значение.

Промпты для отладки и анализа кода

Давайте вручную внесём правки в код из Таблицы 18. Теперь он должен запрашивать у пользователя префикс для имени файла и записывать этот префикс заглавными буквами. Взгляните на пример в Фрагменте *3 — но вот незадача, теперь код выдаёт ошибки Python!

Python

```
import os
import shutil

имя_папки= input("Введите имя папки: ")
prefix= input("Введите строку для добавления к имени файла: ") text =
toUpperCase(prefix)

if not os.path.isdir(имя_папки):
    print("Папка не существует.") exit(1)

файлы= os.listdir(имя_папки)

for file in files:
    new_filename= f"{text}_{file}"

shutil.move(os.path.join(имя_папки, файл),
os.path.joi(имя_папки, имя_нового_файла))

print("Файлы успешно переименованы.")
```

Фрагмент 3. Некорректный Python-скрипт

Ой! Похоже, тут ошибка:

Приведенный ниже код на языке Python выдает ошибку:

```
Traceback (последний вызов):
  Файл "/Users/leeboonstra/Documents/test_folder/rename_files.py", строка 7, in
  <module>
    text= toUpperCase(prefix)
NameError: имя 'toUpperCase' не определено
```

Фрагмент 4. Я сломал Python-код

Давайте попробуем попросить БЯМ помочь с отладкой и проверкой кода.
Загляните в Таблицу 19:

Имя	1_python_debug_code		
Цель	Написать промпт для отладки и проверки кода		
	Python.		
Модель	gemini-pro		
Температура	0.1	Лимит токенов	1024
Топ-K	N/A	Топ-P	1
Промпт	Приведенный ниже код на языке Python выдает ошибку: Файл "/" <pre>Traceback (последний вызов) : Users/leeboonstra/Documents/test_folder/rename_files.py", строка 7, in <module> text= toUpperCase(prefix) NameError: имя 'toUpperCase' не определено</pre> Исправь, что не так, и объясни, как я могу улучшить код. <pre>`python import os import shutil # Получи имя папки от пользователя имя_папки= input("Введите имя папки: ") prefix= input("Введите строку для добавления к имени файла: ") text = toUpperCase(prefix) # Проверь, существует ли папка. if not os.path.isdir(имя_папки): print("Папка не существует.") exit(1) # Получи список всех файлов в папке files= os.listdir(folder_name) # Итерация по файлам для файла в files:</pre> Продолжение на следующей странице...		

Промпт	<pre># Создай новое имя файла с префиксом "draft_" new_filename= f"{text}_{file}" # Перемести файл на новое имя shutil.move(os.path.join(folder_name, file), os.path.join(folder_name, new_file_name)) # Выведи сообщение об успешной операции print("Файлы переименованы успешно.") '''</pre>
Выходные данные	Код вызывает функцию `toUpperCase` для преобразования `префикса` в верхний регистр, но эта функция не определена. Чтобы решить эту проблему, вы можете использовать метод `upper()` класса string, который переводит заданную строку в верхний регистр. Модифицированный код показан ниже: <pre>```python import os import shutil # Получите имя папки от пользователя folder_name= input("Введите имя папки: ") prefix= input("Введите строку для добавления к имени файла: ") text = prefix.upper() # Проверьте, существует ли папка. if not os.path.isdir(имя_папки): print("Папка не существует.") exit(1) # Получите список всех файлов в папке files= os.listdir(folder_name) # Итерация по файлам для файла в files: # Создайте новое имя файла с префиксом "draft_" new_filename= f"{text}_{file}" # Переместите файл в новое имя shutil.move(os.path.join(folder_name, file),</pre> Продолжение на следующей странице...

Выходные данные	<pre>os.path.join(folder_name, new_filename)) # Выведите сообщение об успехе print("Файлы переименованы успешно.") ''' Кроме того, в код можно внести еще несколько улучшений:</pre> 1. Расширение файлов в новых именах не сохраняется. Лучше извлечь расширение из исходного имени файла и добавить его к новому имени. 2. В коде предполагается, что в имени папки нет пробелов. Тем не менее, рекомендуется изящно обрабатывать пробелы, заменяя их символами подчеркивания или дефиса. 3. Использование синтаксиса `f` для интерполяции строк более предпочтительно и лаконично, чем традиционный оператор `+`. 4. Код не обрабатывает ошибки, которые могут возникнуть в переименовании. Было бы лучше обернуть вызов `shutil.move` в блок `try...except`, чтобы перехватить все возможные ошибки. Вот улучшенный код с учетом этих предложений: <pre>pythonimport os import shutil # Получите имя папки от пользователя folder_name= input("Введите имя папки: ") prefix= input("Введите строку для добавления к имени файла: ") # Проверьте, существует ли папка. if not os.path.isdir(имя_папки): print("Папка не существует.")exit(1) # Получите список всех файлов в папке files= os.listdir(folder_name) # Итерация по файлам для файла в files:</pre> Продолжение на следующей странице...
-----------------	---

Выходные данные	<pre> # Получить расширение файла ext= os.path.splitext(file)[1] # Создайте новое имя файла с префиксом и расширением new_filename= f"{prefix}_{os.path.splitext(file)[0]}.{ext}" # Постройте полные пути для исходного и нового файлов old_path=os.path.join(folder_name, file) new_path= os.path.join(folder_name, new_filename) # Попробуйте переместить файл try: shutil.move(old_path, new_path) except Exception as e: print(f"Произошла ошибка при перемещении файла {file}: {e}") # Выведите сообщение об успехе print("Файлы переименованы успешно.") ''' Эти улучшения делают код более надежным и гибким, а также облегчают его чтение и понимание... (Ответ был сокращен, потому что он достиг лимита токенов. Попробуйте увеличить лимит токенов, если вам нужен более развернутый ответ).</pre>
-----------------	---

Таблица 19. Пример запроса для отладки и анализа Python-кода

Отлично. Модель не только подсказала, как решить проблему, но и обнаружила другие ошибки в моём коде – и предложила, как их исправить.

В завершении запроса она также дала рекомендации по общему улучшению кода

Что такое мультимодальные промпты?

Запросы кода по-прежнему делаются с помощью обычной большой языковой модели. Мультимодальные промпты — это отдельный подход, который предполагает использование разных типов входных данных для управления моделью, а не только текста. Это могут быть комбинации текста, изображений, аудио, кода и других форматов — всё зависит от возможностей модели и конкретной задачи.

Лучшие практики

Поиск правильного промпта требует экспериментов. Language Studio в Vertex AI — отличное пространство для того, чтобы поиграть с формулировками и протестировать их на разных моделях.

Используйте следующие лучшие практики, чтобы стать настоящим профи в искусстве создания промптов.

Приводите примеры

Самая важная практика — включать в запрос примеры (one-shot / few-shot). Это один из самых эффективных способов, потому что такие примеры служат для модели своего рода обучающими кейсами.

Они показывают, какого результата вы ожидаете, и позволяют модели нацелиться на конкретный стиль, тон и формат ответа. Это как задать эталон — модель адаптирует свою генерацию, чтобы максимально соответствовать вашим ожиданиям.

Настройки с упрощением

Запросы должны быть краткими, ясными и легко понимаемыми как для вас, так и для модели. В качестве правила: если запрос уже вызывает путаницу у вас, скорее всего, он будет труден для понимания и моделью. Старайтесь избегать сложных выражений и не предоставлять лишней информации.

Примеры:

ДО:

Я сейчас в Нью-Йорке, и мне хотелось бы узнать больше о хороших местах. Я с двумя детьми 3-х лет. Куда нам пойти во время отпуска?

ПОСЛЕ:

Действуй как гид для туристов. Опиши лучшие места для посещения в Нью-Йорке (Манхэттен) с 3-летними детьми

Попробуйте использовать глаголы, которые описывают действия. Вот несколько примеров:

Действия:

Действуй, Анализируй, Классифицируй, Сравни, Создавай, Описывай, Определяй, Оценивай, Извлекай, Находи, Генерируй, Идентифицируй, Перечисляй, Измеряй, Организовывай, Парсинг, Выбирай, Прогнозируй, Предоставляй, Ранжируй, Рекомендуй, Возвращай, Извлекай, Переписывай, Выбирай, Показывай, Сортируй, Резюмируй, Переводи, Пиши.

Используйте конкретику относительно выходных данных

Будь конкретным в указаниях относительно желаемого вывода. Краткие инструкции могут быть недостаточными для правильного направления модели или слишком общими. Предоставление конкретных деталей в запросе (через системные или контекстные промпты) помогает модели сосредоточиться на релевантной информации, что улучшает точность общего результата.

Примеры:

ЧТО ДЕЛАТЬ:

Создай блог-пост из 3 параграфов о топ-5 видеоигровых консолей. Блог-пост должен быть информативным и интересным, написан в разговорном стиле.

ЧЕГО ИЗБЕГАТЬ:

Создай блог-пост о видеоигровых консолях.

Используйте инструкции вместо ограничений

Инструкции и ограничения используются в промптинге, чтобы направить генерацию ответа языковой модели.

- Инструкция — это чёткое указание на желаемый формат, стиль или содержание ответа. Она говорит модели, *что* нужно сделать или *что* должно получиться в результате.
- Ограничение — это набор рамок или запретов. Оно указывает, *чего модель не должна делать* или *чего следует избегать*.

Последние исследования показывают, что использование в промптах позитивных инструкций работает эффективнее, чем акцент на ограничениях. Такой подход совпадает с тем, как люди предпочитают получать именно позитивные указания, а не списки запретов.

Инструкции чётко формулируют желаемый результат, тогда как ограничения могут заставлять модель гадать, что разрешено, а что нет. Позитивные инструкции дают больше свободы и стимулируют креативность в рамках заданных правил, в то время как ограничения могут сдерживать потенциал модели. Кроме того, список ограничений иногда может содержать противоречия.

Ограничения всё ещё полезны, но в конкретных случаях — чтобы не допустить генерации вредоносного или предвзятого контента, либо когда нужен жёсткий формат или стиль вывода.

По возможности лучше использовать позитивные инструкции: вместо того чтобы говорить модели, чего делать не надо, лучше чётко указать, что нужно сделать. Это помогает избежать путаницы и повысить точность результата.Э

- **ЧТО ДЕЛАТЬ:**

Создайте блог-пост из одного параграфа о топ-5 видеоигровых консолях. Обсудите только консоль, компанию, которая ее сделала, год и общий объем продаж.

- **ЧЕГО ИЗБЕГАТЬ:**

Создайте блог-пост из одного параграфа о топ-5 видеоигровых консолях. Не перечисляйте названия видеоигр.

Лучше всего начинать с приоритезации инструкций, четко заявив, что вы хотите от модели, и использовать ограничения только в случае необходимости для безопасности, ясности или специфических требований. Экспериментируйте и пробуйте разные комбинации инструкций и ограничений, чтобы найти оптимальный подход для ваших задач, и документируйте это.

Контроль максимальной длины токенов

Для контроля длины ответа, сгенерированного БЯМ, можно либо установить максимальный лимит токенов в конфигурации, либо явно запросить определенную длину в вашем запросе. Например:
"Объясни квантовую физику в сообщении длиной с твит"

Использование переменных в запросах

Чтобы повторно использовать запросы и сделать их более динамичными, используйте переменные в запросах, которые могут изменяться в зависимости от различных входных данных. Например, как показано в Таблице 20, запрос, который предоставляет факты о городе. Вместо того чтобы жестко прописывать название города в запросе, используйте переменную. Переменные экономят ваше время и усилия, позволяя избежать повторений. Если вам нужно использовать одну и ту же информацию в нескольких запросах, вы можете сохранить её в переменной, а затем ссылаться на эту переменную в каждом запросе. Это имеет смысл, особенно при интеграции запросов в собственные приложения.

Промпт	ПЕРЕМЕННЫЕ {город} = "Амстердам" ПРОМПТ Вы - . Расскажите мне один факт о городе: {город}
Выходные данные	Амстердам - прекрасный город, полный каналов, мостов и узких улочек. Это прекрасное место для посещения благодаря богатой истории, культуре и ночной жизни.

Таблица 20. Использование переменных в промптах

Экспериментируйте с форматами ввода и стилями письма

Разные модели, конфигурации моделей, форматы запросов, выбор слов и отправка могут привести к различным результатам. Поэтому важно экспериментировать с атрибутами запроса, такими как стиль, выбор слов и тип запроса (zero shot, few shot, system prompt).

Например, запрос с целью сгенерировать текст о революционной игровой консоли Sega Dreamcast может быть сформулирован как вопрос, утверждение или инструкция, что приведет к разным результатам:

- Вопрос: Что такое Sega Dreamcast и почему это была такая революционная консоль?
- Утверждение: Sega Dreamcast — это игровая приставка шестого поколения, выпущенная компанией Sega в 1999 году. Это...
- Инструкция: Напишите один абзац, в котором опишите консоль Sega Dreamcast и объясните, чем она была революционной.

При few-shot-промптинге в задачах классификации варьируйте примеры из разных классов

В целом, порядок примеров в few-shot-промптах не должен иметь большого значения. Однако при выполнении задач классификации важно перемешивать возможные классы ответов в примерах. Это необходимо, потому что иначе модель может переобучиться на конкретном порядке примеров. Перемешивая возможные классы ответов, вы обеспечите, чтобы модель училась распознавать ключевые особенности каждого класса, а не просто запоминала порядок примеров. Это приведет к более надежной и обобщенной производительности на новых данных.

Хорошее правило для начала — начать с 6 примеров в few-shot и тестировать точность с этого момента.

Адаптируйтесь к обновлениям моделей

Важно отслеживать изменения в архитектуре моделей, добавление данных и новых возможностей. Пробуйте новые версии моделей и адаптируйте свои запросы, чтобы лучше использовать новые функции моделей. Инструменты, такие как Vertex AI Studio, отлично подходят для хранения, тестирования и документирования различных версий ваших запросов.

Экспериментируйте с форматами вывода

Помимо формата ввода, стоит поэкспериментировать и с форматом вывода. Для некреативных задач — таких как извлечение, выбор, разбор, сортировка, ранжирование или категоризация данных — попробуйте запрашивать результат в структурированном виде, например в формате JSON или XML.

Возврат данных в виде JSON-объектов даёт ряд преимуществ. В реальных приложениях вам не придётся вручную создавать эту структуру — модель сразу может вернуть данные в отсортированном виде (особенно удобно при работе с датами и временем). Но главное — запрос на вывод в формате JSON заставляет модель соблюдать структуру и снижает риск галлюцинаций (выдуманных данных).

Итак, основные преимущества использования JSON для вывода:

- Ответы приходят всегда в одном стиле
- Фокус на тех данных, которые вы хотите получить

- Меньше шансов на галлюцинации
- Модель понимает связи между элементами
- Вы получаете данные с типами
- Их можно отсортировать

Пример структурированного вывода приведён в таблице 4 в разделе про few-shot промптинг.

Исправление JSON

Хотя формат JSON даёт множество плюсов при работе с данными, у него есть и свои минусы. Его структурированность удобна для обработки и интеграции в приложения, но требует больше токенов, чем обычный текст. Это означает более долгую генерацию и более высокую стоимость. Кроме того, из-за «многословности» JSON легко может занять весь лимит вывода — особенно если генерация обрывается из-за ограничения по количеству токенов. В результате можно получить неполный JSON с отсутствующими закрывающими скобками, что делает его непригодным для использования.

К счастью, есть инструменты, которые помогают с этим справиться. Например, библиотека `json-repair` (доступна в PyPI) умеет автоматически восстанавливать неполные или повреждённые JSON-объекты. Это особенно полезно при работе с JSON, сгенерированным языковой моделью, когда есть риск обрезки ответа.

Работа со схемами

Использовать структурированный JSON для вывода — отличное решение, и в этой статье мы не раз это подчеркивали. Но что насчёт ввода? Хотя JSON прекрасно подходит для организации ответа от БЯМ, он может быть не менее полезен и для структурирования входных данных. Здесь в игру вступают JSON-схемы.

JSON-схема задаёт ожидаемую структуру и типы данных во входном JSON. По сути, вы даёте модели чёткий план того, что она должна получить на вход — это помогает ей сосредоточиться на нужной информации и снижает риск того, что она что-то поймёт неправильно. Более того, схемы позволяют задать связи между данными, а при необходимости — даже «научить» модель учитывать время, добавив поля с датой или таймстампами в нужном формате.

Вот простой пример:

Допустим, вы хотите сгенерировать описания товаров для каталога интернет-магазина. Вместо того чтобы просто писать свободный текст о продукте, вы задаёте структуру — например, с помощью JSON-схемы, в которой описаны атрибуты товара.

```
{
  "тип": "object",
  "properties": {
    "name": { "type": "string", "description": "Название продукта" }, "category":
    { "type": "string", "description": "Категория товара" }, "цена": { "type":
    "число", "формат": "float", "description": "Продукт
цена" },
    "features": {
      "type": "array",
      "items": { "type": "string" }, "description":
      "Ключевые особенности продукта".
    },
    "release_date": { "type": "string", "format": "дата", "описание": "Дата
выпуска продукта" }.
  },
}
```

Фрагмент 5. Определение схемы структурированного вывода

Затем вы можете передать фактические данные о продукте в виде JSON-объекта, соответствующего этой схеме.

```
{
  "name": "Беспроводные наушники",
  "категория": "Электроника", "цена":
  99.99,
  "features": ["Шумоподавление", "Bluetooth 5.0", "20-часовое время автономной
  работы"],
  "release_date": "2023-10-27"
```

Фрагмент 6. Как БЯМ возвращает данные в структурированном виде

Если на этапе подготовки данных вы не просто передаёте модели длинные документы, а даёте структуру (схему) и конкретные данные, БЯМ получает чёткое понимание, с чем работает — например, какие у продукта характеристики, когда он вышел и что важно для описания. Это заметно повышает точность и уместность финального текста.

Такой структурированный ввод помогает модели фокусироваться на ключевых полях, а не блуждать в потоке информации. Особенно это ценно, когда вы обрабатываете большие объёмы данных или интегрируете БЯМ в сложные системы — от каталогов до внутренних бизнес-приложений.

Экспериментируйте вместе с другими специалистами по промптингам

Если вам нужно разработать эффективный промпт, стоит привлечь несколько человек для эксперимента. Следуя лучшим практикам (описанным в этой главе), вы заметите, как результаты будут различаться в зависимости от подхода каждого участника.

Лучшие практики для CoT (Цепочки размышлений)

Для промптинга с Цепочкой размышлений (CoT) важно размещать ответ после рассуждений, потому что генерация рассуждений изменяет токены, которые модель получает при предсказании окончательного ответа.

При использовании CoT и самосогласованности необходимо извлечь окончательный ответ из промпта, отделив его от рассуждений. Для CoT-промптинга установите температуру на 0.

Цепочка размышлений основана на жадном декодировании, когда предсказывается следующее слово в последовательности на основе наибольшей вероятности, присвоенной языковой моделью. В общем случае, при использовании рассуждений для получения окончательного ответа, скорее всего, будет только один правильный ответ. Поэтому температура должна всегда быть установлена на 0.

Фиксируйте разные подходы к формулировке промптов

Этот совет уже упоминался в этой главе, но важно повторить: документируйте все свои попытки создания промптов, чтобы со временем понять, что сработало, а что нет.

Выходные данные промптов могут отличаться в зависимости от модели, настроек выборки и даже разных версий одной и той же модели. Более того, даже при одинаковых промптах для одной и той же модели могут возникать небольшие различия в форматировании предложений и выборе слов. Как уже упоминалось, если два токена имеют одинаковую предсказанную вероятность, то для их выбора может применяться случайный метод. Это может повлиять на предсказание последующих токенов.

Мы рекомендуем создать таблицу в Google Sheets с шаблоном, подобным Таблице 21. Преимущества такого подхода заключаются в том, что у вас будет полный архив, который поможет вам вернуться к вашей работе с промптами, когда это станет необходимым — например, чтобы продолжить работу в будущем (вы будете удивлены, как много можно забыть за короткий перерыв), протестировать производительность промпта на разных версиях модели или помочь устранить ошибки в будущем.

Кроме полей в этой таблице, полезно отслеживать версию промпта (итерацию), поле для оценки результата (OK/NOT OK/SOMETIMES OK), а также поле для сбора отзывов. Если вам повезло использовать Vertex AI Studio, сохраняйте свои промпты (с тем же названием и версией, как указано в документации) и отслеживайте ссылку на сохранённый промпт в таблице. Так вы всегда будете иметь возможность в один клик снова запустить свои промпты.

При работе с системой, использующей расширенное извлечение данных (RAG), также полезно зафиксировать специфические аспекты этой системы, которые влияют на то, какой контент был вставлен в промпт, включая запрос, настройки фрагментов, результаты фрагментов и другую информацию.

Как только вы почувствуете, что промпт близок к совершенству, перенесите его в кодовую базу проекта. В кодовой базе храните промпты в отдельном файле от кода, чтобы их было проще поддерживать. В идеале, ваши промпты должны быть частью операционализированной системы, и, как инженер промптов, вам следует опираться на автоматические тесты и процедуры оценки, чтобы понять, насколько хорошо ваш промпт адаптирован к задаче.

Настройка промптов — это итеративный процесс. Разрабатывайте и тестируйте различные промпты, анализируйте и документируйте результаты. Улучшайте свои промпты на основе производительности модели. Продолжайте экспериментировать, пока не получите желаемый результат. Когда вы меняете модель или конфигурацию модели, возвращайтесь и продолжайте экспериментировать с ранее использованными промптами.

Имя	[название и версия вашей программы]		
Цель	[Объяснение цели этой попытки в одном предложении]		
Модель	[название и версия используемой модели]		
Температура	[значение между 0 и 1]	Лимит жетонов	[номер]
Топ-К	[номер]	Топ-Р	[номер]
Промпт	[Напишите полный текст промпта] .		
Выходные данные	[Записать выходные данные или несколько вариантов выходных данных]		

Таблица 21. Шаблон для документирования промптов

Резюме

Этот документ посвящен инженерии промптов. Мы рассмотрели различные техники создания промптов, такие как:

- Нулевой промптинг (Zero prompting)
- Промптинг с несколькими примерами (Few shot prompting)
- Системный промптинг (System prompting)
- Ролевой промптинг (Role prompting)
- Контекстуальный промптинг (Contextual prompting)
- Промптинг с шагом назад (Step-back prompting)
- Цепочка рассуждений (Chain of thought)
- Самосогласованность (Self consistency)
 - Дерево размышлений (Tree of thoughts)

- ReAct

Мы даже рассмотрели способы автоматизации промптов.

Далее в статье рассматриваются ключевые проблемы, связанные с использованием искусственного интеллекта — в частности, те, которые могут возникнуть при недостаточно продуманных промптах. В завершение мы собрали лучшие практики, которые помогут вам эффективнее работать с промптами.

Примечания

1. Google, 2023, Gemini by Google. Доступно по адресу: <https://gemini.google.com>.
2. Google, 2024, Gemini for Google Workspace Prompt Guide. Available at: <https://inthecloud.withgoogle.com/gemini-for-google-workspace-prompt-guide/dl-cd.html>.
3. Google Cloud, 2023, Введение в промптинг. Доступно по адресу: <https://cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/introduction-prompt-design>.
4. Google Cloud, 2023, тело запроса текстовой модели: методы выборки Top-P и Top-K. Доступно по адресу: https://cloud.google.com/vertex-ai/docs/generative-ai/model-reference/text#request_body.
5. Wei, J., et al., 2023, Zero Shot – Fine Tuned language models are zero shot learners. Доступно по адресу: <https://arxiv.org/pdf/2109.01652.pdf>.
6. Google Cloud, 2023, Google Cloud Model Garden. Доступно по адресу: <https://cloud.google.com/model-garden>.
7. Браун, Т., и др., 2023, Few Shot – языковые модели для малоопытных учеников. Available at: <https://arxiv.org/pdf/2005.14165.pdf>.
8. Чжэн, Л., и др., 2023, Сделай шаг назад: Evoking Reasoning via Abstraction in Large Language Models. Доступно по адресу: <https://openreview.net/pdf?id=3bq3jsvcQ1>
9. Wei, J., et al., 2023, Chain of Thought Prompting. Available at: <https://arxiv.org/pdf/2201.11903.pdf>.
10. Google Cloud Platform, 2023, Chain of Thought и React. Доступно по адресу: https://github.com/GoogleCloudPlatform/generative-ai/blob/main/language/prompts/examples/chain_of_thought_react.ipynb.
11. Wang, X., et al., 2023, Self Consistency Improves Chain of Thought reasoning in language models. Доступно по адресу: <https://arxiv.org/pdf/2203.11171.pdf>.
12. Яо, С., и др., 2023, Древо мыслей: Deliberate Problem Solving with Large Language Models. Доступно по адресу: <https://arxiv.org/pdf/2305.10601.pdf>.
13. Яо, С., и др., 2023, ReAct: Synergizing Reasoning and Acting in Language Models. Available at: <https://arxiv.org/pdf/2210.03629.pdf>.
14. Google Cloud Platform, 2023, Предварительное наведение справок: Chain of Thought and React. Доступно по адресу: https://github.com/GoogleCloudPlatform/applied-ai-engineering-samples/blob/main/genai-on-vertex-ai/advanced_prompting_training/cot_react.ipynb.
15. Zhou, C., et al., 2023, Automatic Prompt Engineering – Large Language Models are Human–

Настройка промптов

Level Prompt Engineers. Доступно по адресу: <https://arxiv.org/pdf/2211.01910.pdf>.